

Detailed Design Plan for ML Processing Pipeline

Overview

We want to build a production-ready pipeline for extracting and matching tables from scanned images of record books and corresponding unsanctioned text files. This pipeline should:

1. Ingest input data from AWS S3 (organized by college folders).
 2. Convert PDF pages into images using a custom script.
 3. Process the data and extract tables using AWS Textract.
 4. Match extracted tables with unsanctioned tables using a BLEU score-based comparison.
 5. Output the results (matched and unmatched tables, accuracy reports, processed images, and metadata) back to S3.
 6. Provide an API using FastAPI with routes for input submission, output retrieval, monitoring, and error logging.
 7. Use Celery for background task execution and Redis as the message broker.
-

Step-by-Step Pipeline Design

1. Input Data Structure

- **Main Folder in S3:** `input-data/`
 - Each sub-folder represents a college.
 - Each college folder contains:
 - `record-book.pdf`: The PDF file that contains scanned pages of the record book.
 - `unsan.txt`: The unsanctioned tables text file for that college.
 - `config.json`: Configuration file for AWS Textract parameters.

Example structure:

```
Unset
input-data/
├── college_1/
│   ├── record-book.pdf
│   ├── unsan.txt
│   └── config.json
├── college_2/
│   ├── record-book.pdf
│   ├── unsan.txt
│   └── config.json
└── ...
```

2. Data Ingestion

- Use **boto3** to connect to AWS S3.
- List all college folders in the main input folder.
- For each college folder:
 1. Download `record-book.pdf`, `unsan.txt`, and `config.json` to a local temporary directory.
 2. Convert `record-book.pdf` into images (one image per page) using a custom script and save them in an `images/` folder.

3. Table Extraction and Matching

Table Extraction

- For each image in the `images/` folder, use AWS Textract to detect tables based on the parameters in `config.json`.
- Extract relevant table details (bounding box, text content, etc.).
- Save all detected tables in a folder named `all_tables_detected` under the college folder.

Table Matching

- Use the BLEU score-based approach to match tables from images with the tables in `unsan.txt`.
- Save matched tables with bounding boxes drawn in a folder named `matched_tables_drawn` under the college folder.
- Maintain a `dump` folder for each college, containing:
 - Metadata about the detected tables (bounding boxes, extracted text, etc.).
 - Matching results (BLEU scores, matched table IDs, unmatched tables, etc.).
 - Page-wise and overall matching accuracy.

4. Output Data Structure

- Results will be uploaded back to S3 in a separate output folder:

Unset

`output-data/`

```
├─ college_1/
|   ├─ images/
|   ├─ matched_tables_drawn/
|   ├─ json_files/
|   ├─ dump/
|   |   ├─ textract_tables/
|   |   ├─ matched_tables/
|   |   └─ unmatched_tables/
|   └─ matching_evaluation.txt
├─ college_2/
|   ├─ images/
|   ├─ matched_tables_drawn/
|   ├─ json_files/
|   ├─ dump/
|   |   └─ textract_tables/
```

```
| | |— matched_tables/
| | |— unmatched_tables/
| |— matching_evaluation.txt
|— ...
```

5. Monitoring and Logging

Monitoring

- Implement a monitoring feature using FastAPI.
- Provide a dashboard that shows:
 - Current college being processed.
 - Number of images processed so far.
 - Total number of images.
 - Overall accuracy and page-wise accuracy.

Logging

- Use Python's built-in logging module to log:
 - Start and completion of processing for each college.
 - Errors encountered during processing.
 - Detailed error messages with stack traces for debugging.
- Store logs in a separate folder on S3 (`logs/`) and also save an `error_log.txt` file in the `dump` folder for each college.

6. API Design with FastAPI

Routes

1. **POST /submit-job**
 - Trigger a new job for processing a college folder.
 - Input: College folder name.
 - Response: Job ID.
2. **GET /job-status/{job_id}**
 - Check the status of a job.
 - Response: Current status (e.g., running, completed, failed), college being processed, progress percentage, and accuracy.
3. **GET /results/{college_name}**
 - Retrieve the results for a specific college.

- Response: Download link for the output data.
4. **GET /logs/{college_name}**

- Retrieve error logs for a specific college.

7. Background Task Execution with Celery

- Use **Celery** to run the processing jobs in the background.
 - Use **Redis** as the message broker.
 - Celery worker processes will handle the heavy lifting of downloading files, processing images, and uploading results.
-

Deployment Plan

1. Infrastructure

- **AWS S3**: For input and output data storage.
- **AWS Textract**: For table extraction.
- **Redis**: For Celery message brokering.
- **EC2 / ECS / Fargate**: For deploying the FastAPI app and Celery workers.

2. CI/CD Pipeline

- Use **GitHub Actions** or **AWS CodePipeline** for continuous integration and deployment.
- Steps:
 1. Linting and testing.
 2. Build Docker image.
 3. Push Docker image to AWS ECR.
 4. Deploy to ECS / Fargate.

3. Error Handling and Retry Logic

- Implement retry logic in Celery for transient errors (e.g., network issues).
- Use exponential backoff for retries.
- Log all errors and retries.

4. Security Considerations

- Use AWS IAM roles with least privilege for accessing S3 and Textract.
- Secure Redis with authentication and TLS.
- Use HTTPS for the FastAPI app.

Sequence Diagram

